

Question 1: Code Tracing - Branching, Loops, and Functions

Question 1: Code-Tracing

$x = \text{mystery}(4)$

Variables	Values
total	0

for loop: $i = 1, 2, 3, 4$

$i = 1$

$1 \% 2 \neq 0$

$1 \% 3 \neq 0$

$\text{total} = -1$

$i = 2$

$2 \% 2 == 0$

$\text{total} = 1 + 2 = 1$

$i = 3$

$3 \% 2 \neq 0$

$3 \% 3 == 0$

$\text{total} = 1 + 2(3) = 7$

$i = 4$

$4 \% 2 == 0$

$\text{total} = 7 + 4 = 11$

return

$x = 11$

$'total' = 0$

$'i' = 1$

$'total' = 1$

$'i' = 2$

$'total' = 1$

$'i' = 3$

$'total' = 7$

$'i' = 4$

$'total' = 11$

$'x' = 11$

create-list([11, 3, 5])

lst = [11, 3, 5]

new-lst = []

for loop: item = 11, 3, 5

item = 11

$11 \% 3 \neq 0$

new-lst = [12]

item = 3

$3 \% 3 = 0$

new-lst = [12, 6]

item = 5

$5 \% 3 \neq 0$

new-lst = [12, 6, 6]

— returned

x = 11

y = [12, 6, 6]

output

x: 11

y: [12, 6, 6]

Question 2: Programming - Permutations

Answer:

```
def isPermutation(s: str, t: str) -> bool:
    try:
        if len(s) != len(t):
            return False

        sMap = {}
        tMap = {}

        for c in s:
            sMap[c] = sMap.get(c, 0) + 1

        for c in t:
            tMap[c] = tMap.get(c, 0) + 1

        return sMap == tMap

    except TypeError:
        print("Error: Both inputs must be strings.")
        return False

# Alternate version: uses one dictionary
def isPermutationAlternate(s1: str, s2: str) -> bool:
    try:
        if len(s1) != len(s2):
            return False

        freq = {}

        for c in s1:
            freq[c] = freq.get(c, 0) + 1

        for c in s2:
            if c not in freq or freq[c] == 0:
                return False

            freq[c] -= 1

        return True

    except TypeError:
        print("Error: Both inputs must be strings.")
        return False
```

Question 3: Code Tracing - OOP

1. Output before the error:

Trapezoid: height: 5.5, base1: 7, base2: 9
area: 44.0
Rectangle: height: 4, width: 6
area: None
new height: 8
new area: 64

2. Error:

```
print(f"base1: {tr.__base1}")
```

This produces an `AttributeError` because `__base1` is private. The statements after this line are not executed.

3. Methods called:

- For `tr`: `Trapezoid.calcArea()`
- For `rect`: `Rectangle.calcArea()`

4. Polymorphism:

The same call, `shape.calcArea()`, runs a different overridden method depending on whether `shape` refers to a `Trapezoid` or a `Rectangle`.

Q4: Conceptual Understanding - Data Structures

- a) dictionaries
- b) sets

Q5

Solution:

```
def calculate_averages(grades):
    averages = []
    for student in grades:
        averages.append(round(sum(student) / len(student), 2))
    return averages

def find_highest_grades(grades):
    max_math = grades[0][0]
    max_science = grades[0][1]
    max_english = grades[0][2]

    for student in grades:
        if student[0] > max_math:
            max_math = student[0]
        if student[1] > max_science:
            max_science = student[1]
        if student[2] > max_english:
            max_english = student[2]

    return (max_math, max_science, max_english)

def get_unique_grades(grades):
    unique_grades = set()
    for student in grades:
        for grade in student:
            unique_grades.add(grade)
    return unique_grades

def main():
    grades = [[85, 90, 88], [78, 82, 80], [90, 85, 92], [70, 75, 68]]

    print("Averages:", calculate_averages(grades))
    print("Highest Grades:", find_highest_grades(grades))
    print("Unique Grades:", get_unique_grades(grades))

main()
```

Q6:

Output

0

[6.4, 62.5, 0]

Q7: Tuples

Answer:

Jokic

Giannis

Lebron

Jokic count: 1

Tatum index: 3

Common players: ('Jokic', 'Tatum')

Number of common players: 2

Jokic is at tuple index 0

Tatum is at tuple index 3

Q8:

Answer:

```
def calculate_sum(a,b):  
    sum = 0  
    for i in range(len(a)):  
        if a[i] != b[i]:  
            sum += a[i] * b[i]  
    print(sum)
```

```
def main():  
    calculate_sum(a,b)
```

```
main()
```

Q9:

[09]

matrix = $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

i in range(3)
j in range(3)

i=0:

j=0:

$(0+0) \% 2 == 0$? $\checkmark$
 $\text{matrix}[0][0] = 1 * 2 = 2$

j=1:

$(0+1) \% 2 \neq 0$

j=2:

$(0+2) \% 2 == 0$ $\checkmark$
 $\text{matrix}[0][2] = 3 * 2 = 6$

i=1:

j=0:

$(1+0) \% 2 \neq 0$

j=1:

$(1+1) \% 2 == 0$ $\checkmark$
 $\text{matrix}[1][1] = 5 * 2 = 10$

j=2:

$(1+2) \% 2 \neq 0$

i=2:

j=0:

$(2+0) \% 2 == 0$ $\checkmark$
 $\text{matrix}[2][0] = 7 * 2 = 14$

j=1:

$(2+1) \% 2 \neq 0$ $\times$

j=2:

$(2+2) \% 2 == 0$ $\checkmark$
 $\text{matrix}[2][2] = 9 * 2 = 18$

$\text{matrix}[0][2] = 4$

$\text{matrix}[2][1] = 6$

$\text{matrix}[1][1] = \text{matrix}[0][1] + \text{matrix}[0][2]$
 $= 2 + 4$

$\text{matrix}[1][1] = 6$

for row in matrix:

row = [2, 2, 4]

row = [4, 6, 6]

row = [14, 6, 18]

Output:

[2, 2, 4]

[4, 6, 6]

[14, 6, 18]

Q10:

Solution

```
def clean_sentence(sentence, details_requested):
```

```
    parts = sentence.split()
```

```
    words = []
```

```
    removed_parts = []
```

```
    for part in parts:
```

```
        cleaned = ""
```

```
        removed = ""
```

```
        for ch in part:
```

```
            if ch.isalpha():
```

```
                cleaned += ch
```

```
            else:
```

```
                removed += ch
```

```
    if cleaned != "":
```

```
        words.append(cleaned.lower())
```

```
    if removed != "":
```

```
        removed_parts.append(removed)
```

```
    if details_requested:
```

```
        print(removed_parts)
```

```
    return words
```

```
sentence = "hello!!! world, 123 coffee? yes: coffee world... hello"
```

```
print(clean_sentence(sentence, True))
```

Q11:

Question 11: Code-Tracing

variables	values
sentence	"hello!!! world, 123 coffee? yes: coffee world... hello"
words	['hello!!!', 'world,', '123', 'coffee?', 'yes:', 'coffee', 'world...', 'hello']

for i = 0, 1, 2, 3, 4, 5, 6, 7

i = 0

words[0] = "hello" # stripped of punctuation

after i = 1 - 7

words	['hello', 'world', '123', 'coffee', 'yes', 'coffee', 'world', 'hello']
-------	--

find-positions(words, 'coffee')

build-inverted-index(words)

[index | { }] # empty dictionary!

for i in i = 0, 1, 2, 3, 4, 5, 6, 7

i = 0

word = 'hello'

[index | { 'hello': [] }] → { 'hello': [0] }

after i = 1 - 7

index { 'hello': [0, 7], 'world': [1, 6], '123': [2],
'coffee': [3, 5], 'yes': [4] }

indexes	
result	[3, 5]

 → returned

prints
coffee: [3, 5]

find_positions(words, 'world')
refer to previous build_inverted_index()

indexes	
result	[1, 6]

 → returned

prints
world: [1, 6]

find_positions(words, 'python')

result	None
--------	------

 # .get() on nonexistent key
→ returns False

prints
python: False

Q12:

Solution

```
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species
        self.zoo = ""
        self.__happiness = 50

    def play(self):
        self.__happiness *= 1.5
        return self.__happiness

    def __str__(self):
        if not self.zoo:
            return f"{self.name} is a {self.species}"
        else:
            return f"{self.name} is a {self.species} in {self.zoo}"

class Zoo:
    def __init__(self, name, animals):
        self.name = name
        self.animals = animals
        # assign zoo to existing animals
        for animal in self.animals:
            animal.zoo = self.name

    def add_animal(self, animal):
        self.animals.append(animal)
        animal.zoo = self.name

    def __str__(self):
        s = f"{self.name}:\n"
        for animal in self.animals:
            s += f"    {animal}\n"
        return s

def main():
    bear = Animal('Baloo', 'bear')
    tiger = Animal('Shere Khan', 'tiger')
    print(bear.play())
    print(bear)
    print(tiger)
    my_zoo = Zoo('Our New Zoo Zoo', [bear])
    my_zoo.add_animal(tiger)
    print(my_zoo)

main()
```